LAPORAN TUGAS B:

Membangun API Microservice

Disusun guna memenuhi tugas mata kuliah

II3160 Teknologi Sistem Terintegrasi



Disusun oleh:

Faizal Ali	18224007
Muhammad Azikra Wira Pratama	18224089

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

B1. PocketBase:Backend Open Source Tanpa Coding

Unduh terlebih dahulu PocketBase dengan menjalankan command berikut di PowerShell:

```
wget https://github.com/pocketbase/pocketbase/releases/download/v0.40.5/pocketbase_0.40.5_linux_arm64.zip
```

Setelah itu, unzip file yang sudah didownload dengan command berikut:

```
unzip pocketbase_0.40.5_linux_arm64.zip
```

Jika berhasil, tampilannya akan seperti gambar di bawah.

```
root@armbian:/mnt/flashdisk# unzip pocketbase_0.40.5_linux_arm64.zip
Archive:  pocketbase_0.40.5_linux_arm64.zip
  inflating: CHANGELOG.md
  inflating: LICENSE.md
  inflating: pocketbase
```

Setelah itu cek versi PocketBase dengan command berikut:

```
./pocketbase --version
```

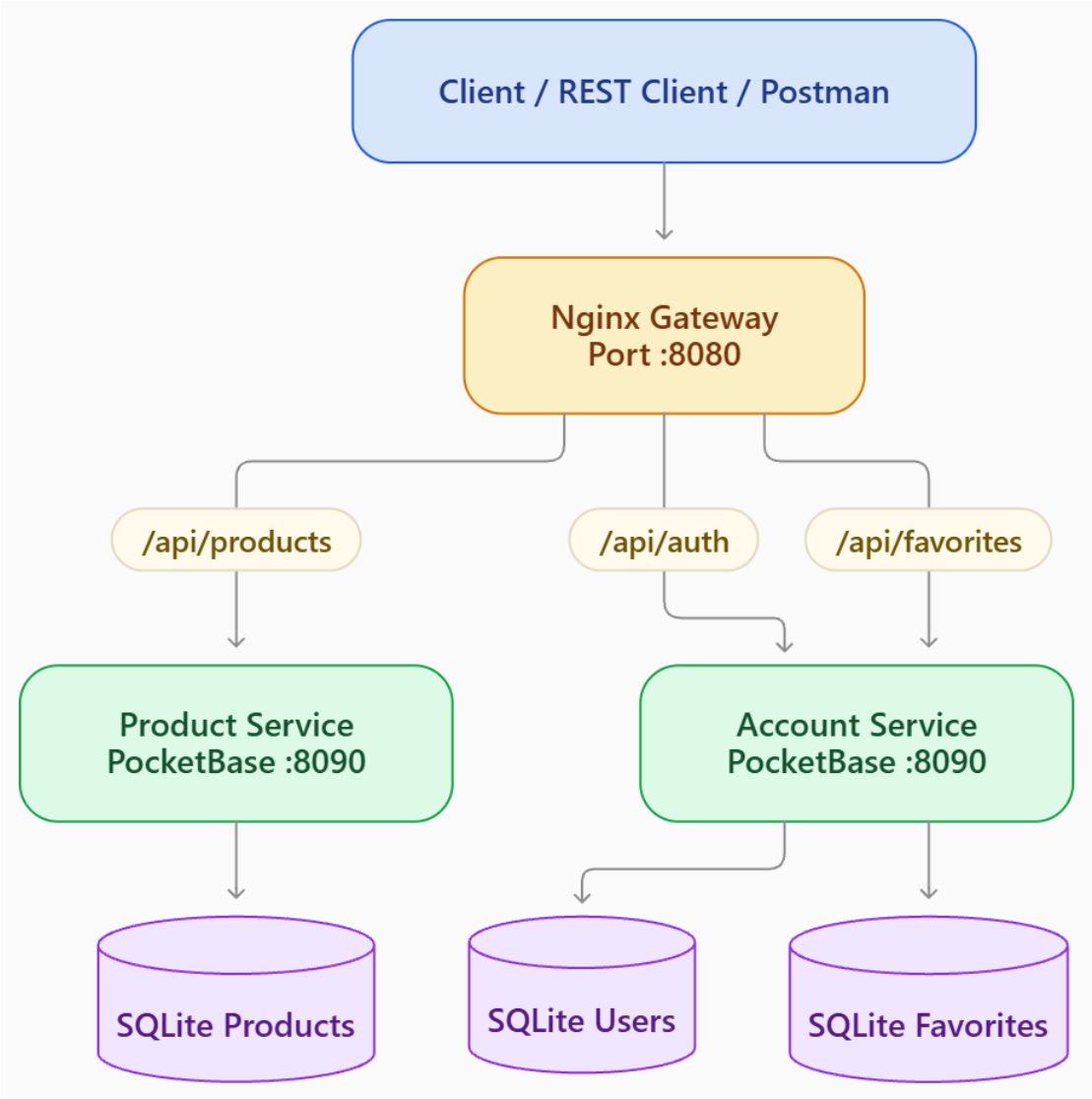
Jika versi 0.40.5 sudah muncul, maka proses instalasi berhasil.

```
root@armbian:/mnt/flashdisk# ./pocketbase --version
pocketbase version 0.40.5
```

B2: Rancangan Service KatalogKu

Service	Isi	Port	Data	Prefix URL (gateway)
Gateway (nginx)	Meneruskan request ke service yang tepat	8080	-	/api/...
product-service	PocketBase: products	8091 (dashboard)	SQLite sendiri	/api/products
account-service	PocketBase: users dan favorites	8092 (dashboard)	SQLite sendiri	/api/auth /api/favorites

B3: Arsitektur & Alur Request



B4: Struktur Folder Proyek

Struktur proyek katalogku-pocketbase memisahkan konfigurasi Docker, Nginx, skema koleksi, dan berkas pengujian. Folder gateway digunakan untuk aturan routing, sedangkan folder pocketbase berisi Dockerfile image backend.

```
katalogku-pocketbase/
├── .postman/
```

```
├── gateway/
│   └── nginx.conf
├── pb_schema/
│   ├── account_collections.json
│   └── product_collections.json
├── pocketbase/
│   └── Dockerfile
├── postman/
│   ├── environments/
│   │   └── New Environment.environment.yaml
│   ├── globals/
│   │   └── workspace.globals.yaml
│   ├── KatalogKu-Cloudflare.postman_environment.json
│   ├── KatalogKu-Lokal.postman_environment.json
│   ├── KatalogKu-PocketBase.postman_collection.json
│   └── KatalogKu-ServerLAN.postman_environment.json
├── .env.example
├── .gitignore
├── api.http
├── docker-compose.yml
├── README.md
├── 18224007_18224089_Pocketbase.pdf
└── seed.http
```

B5: Dockerfile PocketBase

Dockerfile digunakan untuk membangun image PocketBase berdasarkan Alpine Linux. Variabel TARGETARCH menentukan jenis binary PocketBase yang diunduh sesuai arsitektur komputer atau server. Seluruh data persisten ditempatkan pada direktori /pb_data.

```
# Image PocketBase untuk amd64 (VM/laptop) & arm64 (STB)
FROM alpine:3.22
ARG PB_VERSION=0.40.5
# TARGETARCH otomatis diisi Docker: amd64 atau arm64
ARG TARGETARCH
ARG PB_URL=https://github.com/pocketbase/pocketbase/releases/download
```

```
RUN apk add --no-cache ca-certificates wget unzip \
 && wget -q -O /tmp/pb.zip \

"${PB_URL}/v${PB_VERSION}/pocketbase_${PB_VERSION}_linux_${TARGETARCH}.
zip" \
 && unzip /tmp/pb.zip pocketbase -d /usr/local/bin \
 && rm /tmp/pb.zip

EXPOSE 8090
CMD ["pocketbase", "serve", "--http=0.0.0.0:8090", "--dir=/pb_data"]
```

Dengan konfigurasi tersebut, kedua service dapat menggunakan definisi image yang sama, tetapi tetap memiliki volume data yang berbeda.

B6: docker-compose.yml

Docker Compose mengelola tiga container, yaitu product-service, account-service, dan gateway. Product-service dipetakan ke port 8091, account-service ke port 8092, dan Nginx gateway ke port 8080.

Docker compose file berisikan kode berikut:

```
volumes:
  product_data:   # database & file milik product-service
  account_data:   # database & file milik account-service

services:
  product-service:        # PocketBase #1: koleksi products
    build: ./pocketbase
    restart: unless-stopped
    volumes:
      - product_data:/pb_data
    ports:
      - "8091:8090"       # dashboard: http://IP_SERVER:8091/_/

  account-service:        # PocketBase #2: users + favorites
    build: ./pocketbase
    restart: unless-stopped
    volumes:
      - account_data:/pb_data
    ports:
```

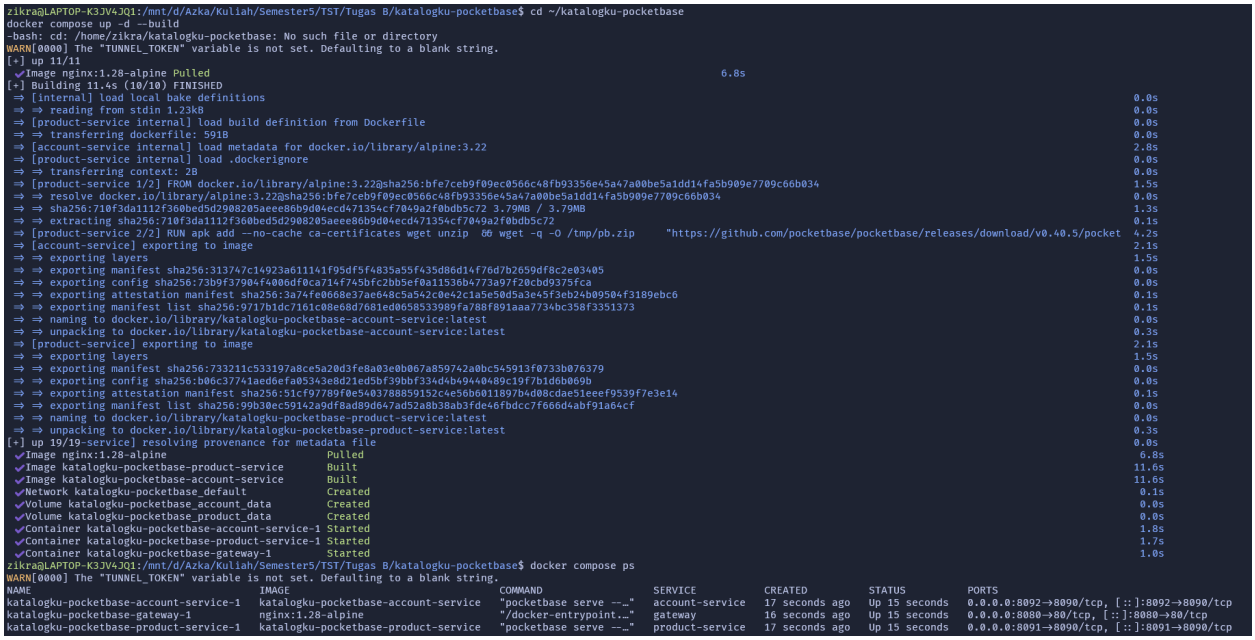
```
    - "8092:8090"        # dashboard: http://IP_SERVER:8092/_/

  gateway:               # satu pintu masuk API
    image: nginx:1.28-alpine
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./gateway/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - product-service
      - account-service

  tunnel:                # opsional: Cloudflare Tunnel
    image: cloudflare/cloudflared:latest
    restart: unless-stopped
    command: tunnel --no-autoupdate run --token ${TUNNEL_TOKEN}
    depends_on:
      - gateway
    profiles: ["tunnel"]
```

Kemudian menjalankan command ini pada WSL:

```
docker compose up -d --build
docker compose ps
```



Output terminal menunjukkan proses build image dan peluncuran layanan melalui Docker Compose. Status layanan diperiksa agar kedua PocketBase dan Nginx dapat digunakan pada tahap selanjutnya.

B7: API Gateway: Memetakan URL ke PocketBase

API Gateway menggunakan Nginx untuk meneruskan request dari satu alamat publik ke dua layanan backend. Pengaturan ini terdapat pada gateway/nginx.conf.

URL Gateaway	Service Tujuan	Path Pocketbase
/api/health	Nginx gateway	Respons JSON {"gateway":"ok"}
/api/products	product-service	/api/collections/products/records
/api/auth/...	account-service	/api/collections/users/...
/api/favorites	account-service	/api/collections/favorites/records

B8: Jalankan & Buat Akun Superuser

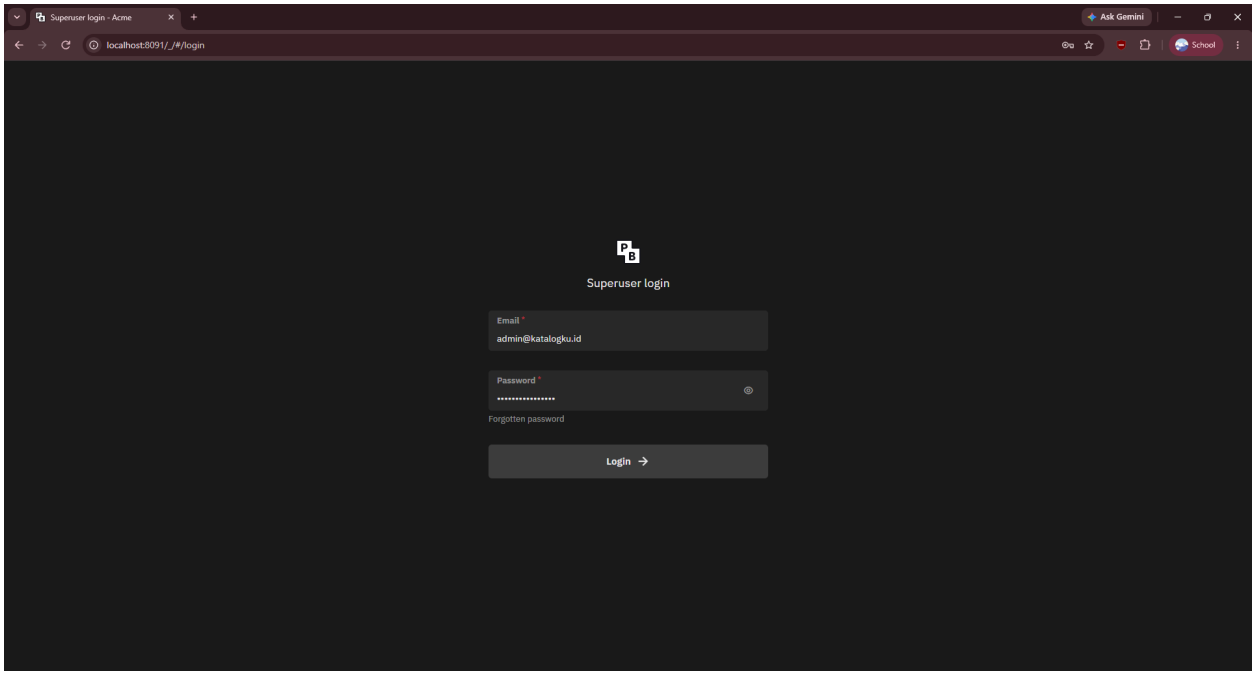
Setelah container berjalan, superuser dibuat untuk product-service dan account-service. Kedua layanan menggunakan database berbeda sehingga masing-masing memerlukan akun administrasi sendiri.

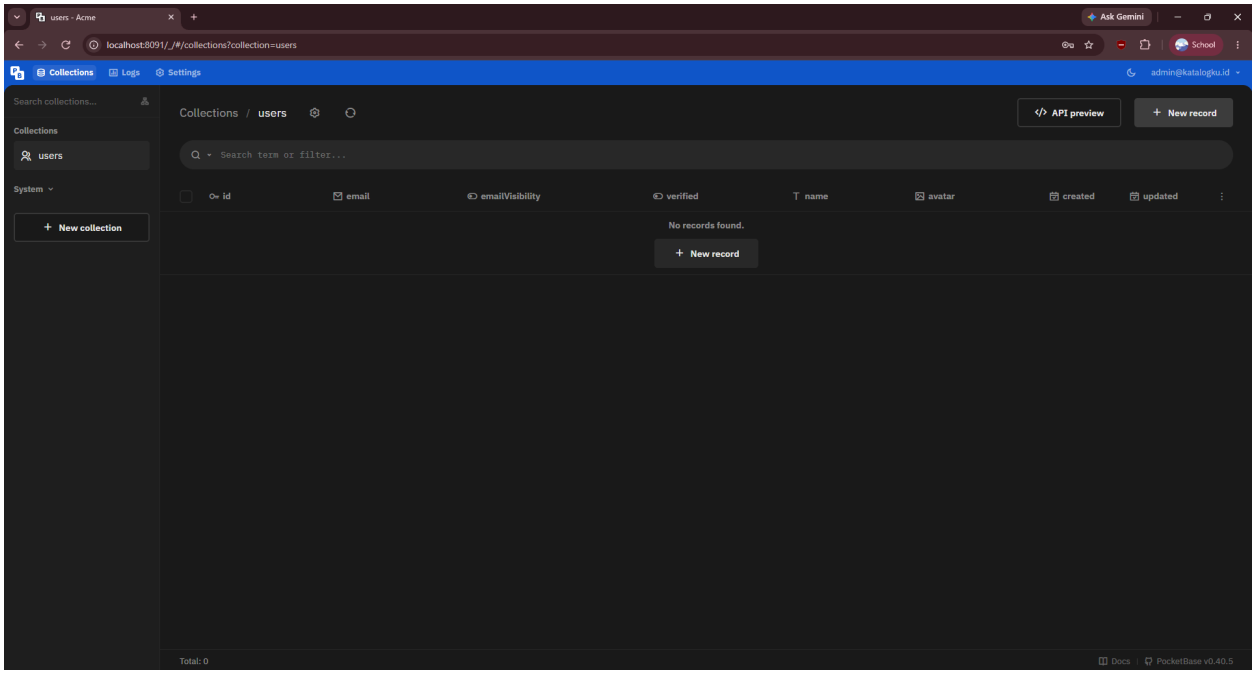
```
docker compose exec product-service \
  pocketbase superuser upsert admin@katalogku.id GantiPassword123
--dir=/pb_data

docker compose exec account-service \
  pocketbase superuser upsert admin@katalogku.id GantiPassword123
--dir=/pb_data
```

```
zikra@LAPTOP-K3JV4JQ1:/mnt/d/Azka/Kuliah/Semester5/TST/Tugas B/katalogku-pocketbase$ docker compose exec product-service \
  pocketbase superuser upsert admin@katalogku.id GantiPassword123 --dir=/pb_data
WARN[0000] The "TUNNEL_TOKEN" variable is not set. Defaulting to a blank string.
Successfully saved superuser "admin@katalogku.id"!
zikra@LAPTOP-K3JV4JQ1:/mnt/d/Azka/Kuliah/Semester5/TST/Tugas B/katalogku-pocketbase$ docker compose exec account-service \
  pocketbase superuser upsert admin@katalogku.id GantiPassword123 --dir=/pb_data
WARN[0000] The "TUNNEL_TOKEN" variable is not set. Defaulting to a blank string.
Successfully saved superuser "admin@katalogku.id"!
```

Dashboard product-service dibuka melalui http://localhost:8091/_/, sedangkan account-service melalui http://localhost:8092/_/. Login administrasi digunakan untuk membuat koleksi dan mengatur API Rules.

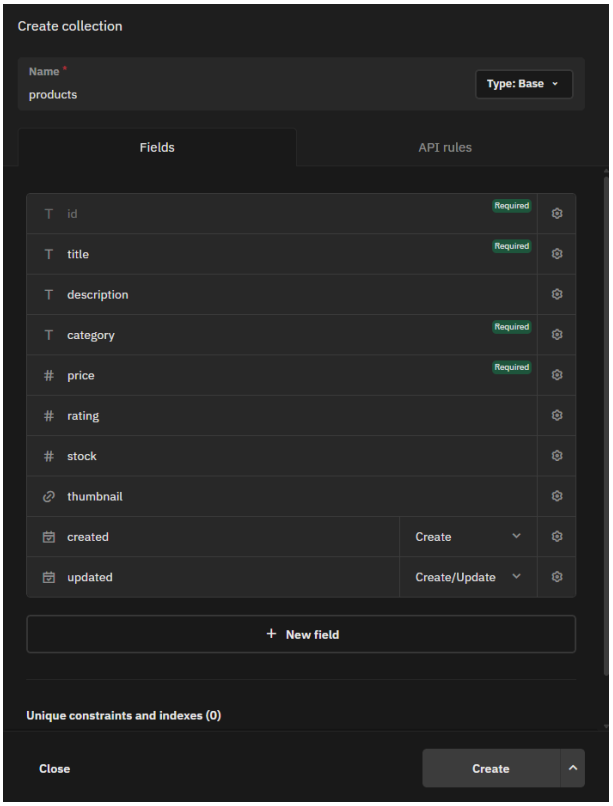




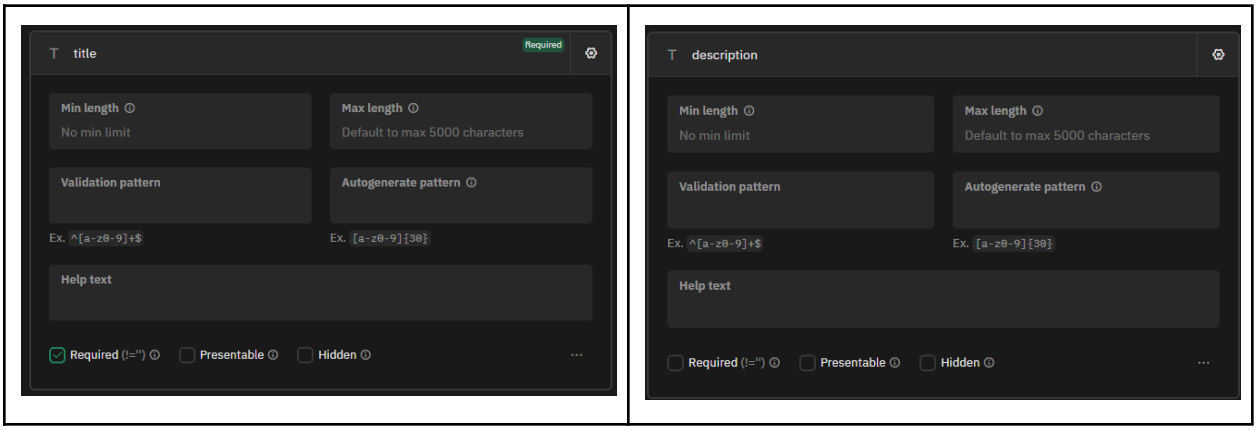
B9: Buat Koleksi Products

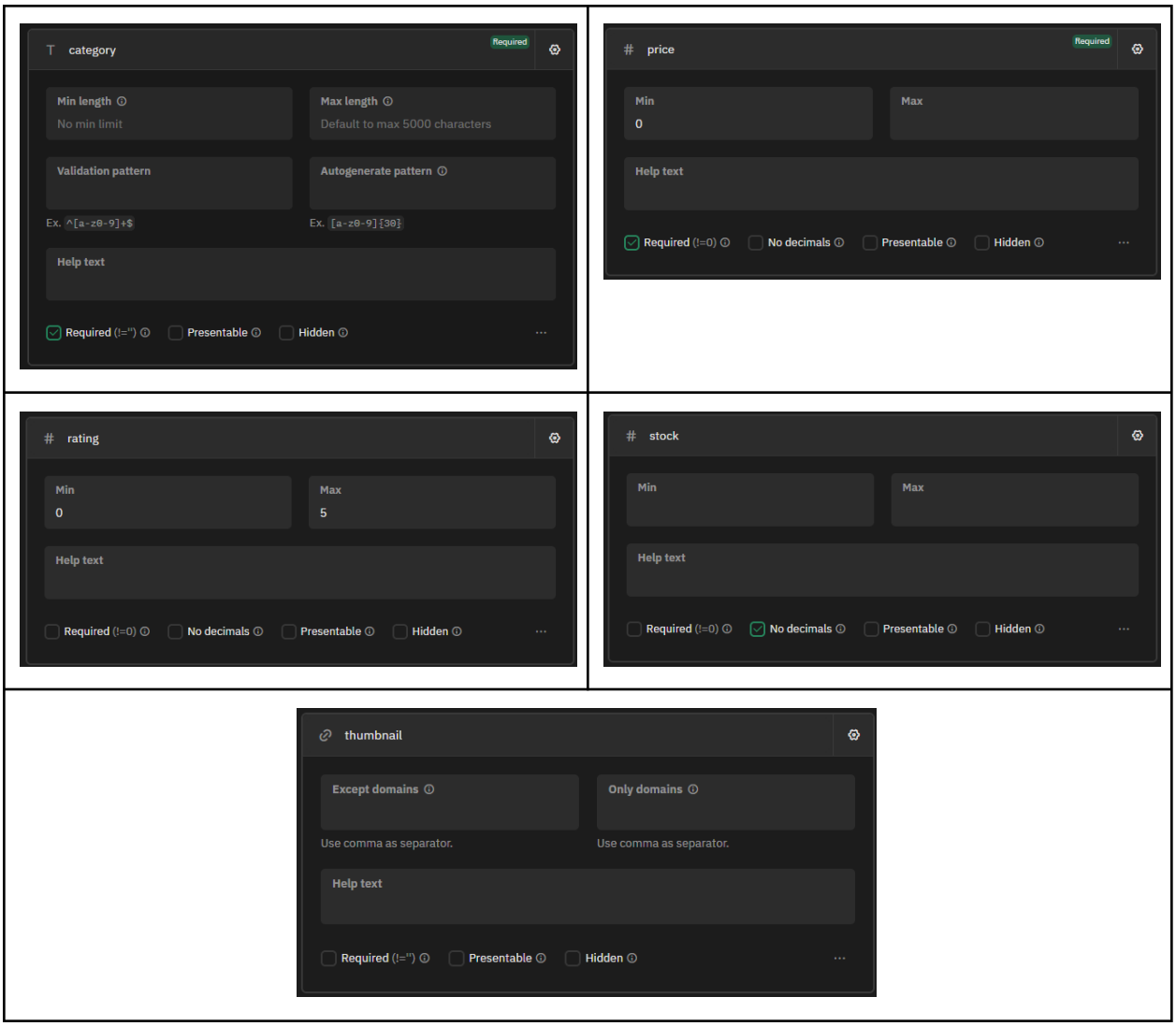
Pada dashboard product-service, koleksi bertipe Base dengan nama products dibuat untuk menyimpan informasi katalog. Field yang disiapkan adalah title, description, category, price, rating, stock, dan thumbnail.

Field	Tipe	Pengaturan
title	Plain text	Required / Nonempty
description	Plain text	Opsional
category	Plain text	Required / Nonempty
price	Number	Required, min 0
rating	Number	Min 0, max 5
stock	Number	No decimals
thumbnail	URL	Tautan gambar

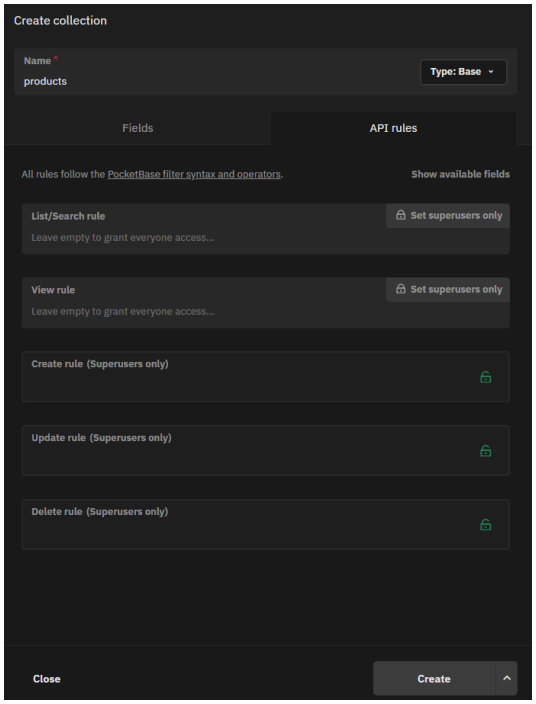


Opsi Nonempty pada materi berarti field wajib diisi. Pada PocketBase, hal tersebut diatur melalui checkbox Required. Field description tetap opsional, sedangkan title dan category harus memiliki nilai.





Hak akses produk diatur pada tab API Rules. Operasi List dan View dapat dibuka untuk publik, sedangkan Create, Update, dan Delete dibiarkan terkunci agar hanya superuser yang dapat mengubah data.



B10: Cara Cepat: Import Skema & Data Contoh

Generate menggunakan seed.http yang berisikan :

```
### Isi data produk awal (opsional, pengganti input manual di dashboard)
### Langsung ke product-service (port 8091), login sebagai superuser.

@pbUrl = http://localhost:8091
@suToken = {{su.response.body.$.token}}

### 0. Login superuser
# @name su
POST {{pbUrl}}/api/collections/_superusers/auth-with-password
Content-Type: application/json

{
  "identity": "admin@katalogku.id",
  "password": "GantiPassword123"
}

### 1. Kemeja Batik Parang
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
```

```
Content-Type: application/json

{
  "title": "Kemeja Batik Parang",
  "description": "Kemeja batik motif parang, bahan katun.",
  "category": "fashion",
  "price": 25,
  "rating": 4.6,
  "stock": 40,
  "thumbnail": "https://picsum.photos/id/21/300"
}

### 2. Kopi Gayo 250 gr
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
Content-Type: application/json

{
  "title": "Kopi Gayo 250 gr",
  "description": "Biji kopi arabika Gayo, sangrai medium.",
  "category": "groceries",
  "price": 9,
  "rating": 4.8,
  "stock": 120,
  "thumbnail": "https://picsum.photos/id/30/300"
}

### 3. Teh Hijau Organik
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
Content-Type: application/json

{
  "title": "Teh Hijau Organik",
  "description": "Teh hijau organik dari kebun Jawa Barat.",
  "category": "groceries",
  "price": 6.5,
  "rating": 4.3,
  "stock": 80,
  "thumbnail": "https://picsum.photos/id/312/300"
```

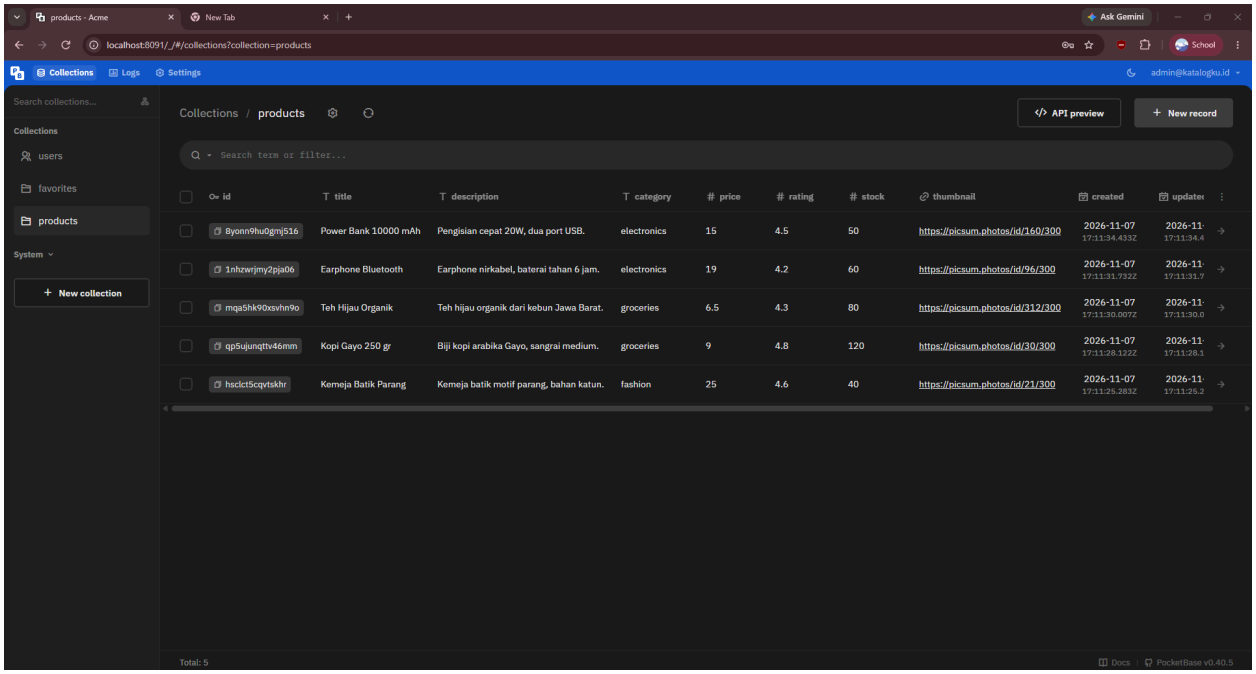
```
}

### 4. Earphone Bluetooth
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
Content-Type: application/json

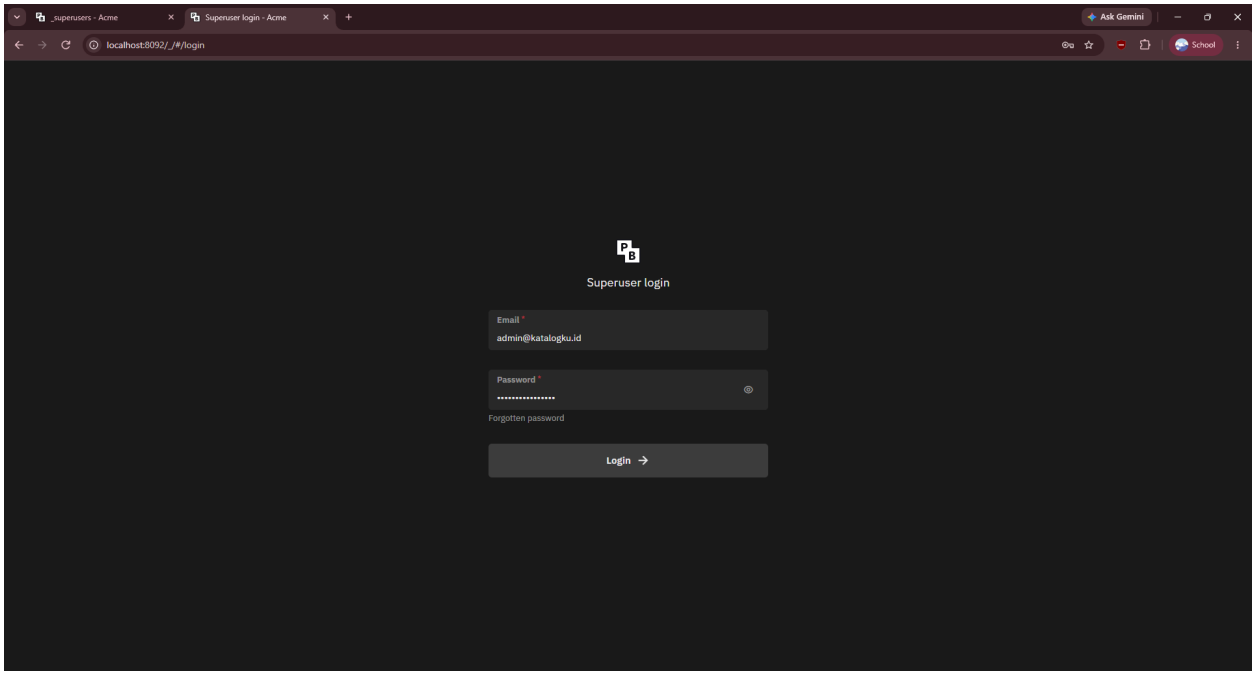
{
  "title": "Earphone Bluetooth",
  "description": "Earphone nirkabel, baterai tahan 6 jam.",
  "category": "electronics",
  "price": 19,
  "rating": 4.2,
  "stock": 60,
  "thumbnail": "https://picsum.photos/id/96/300"
}

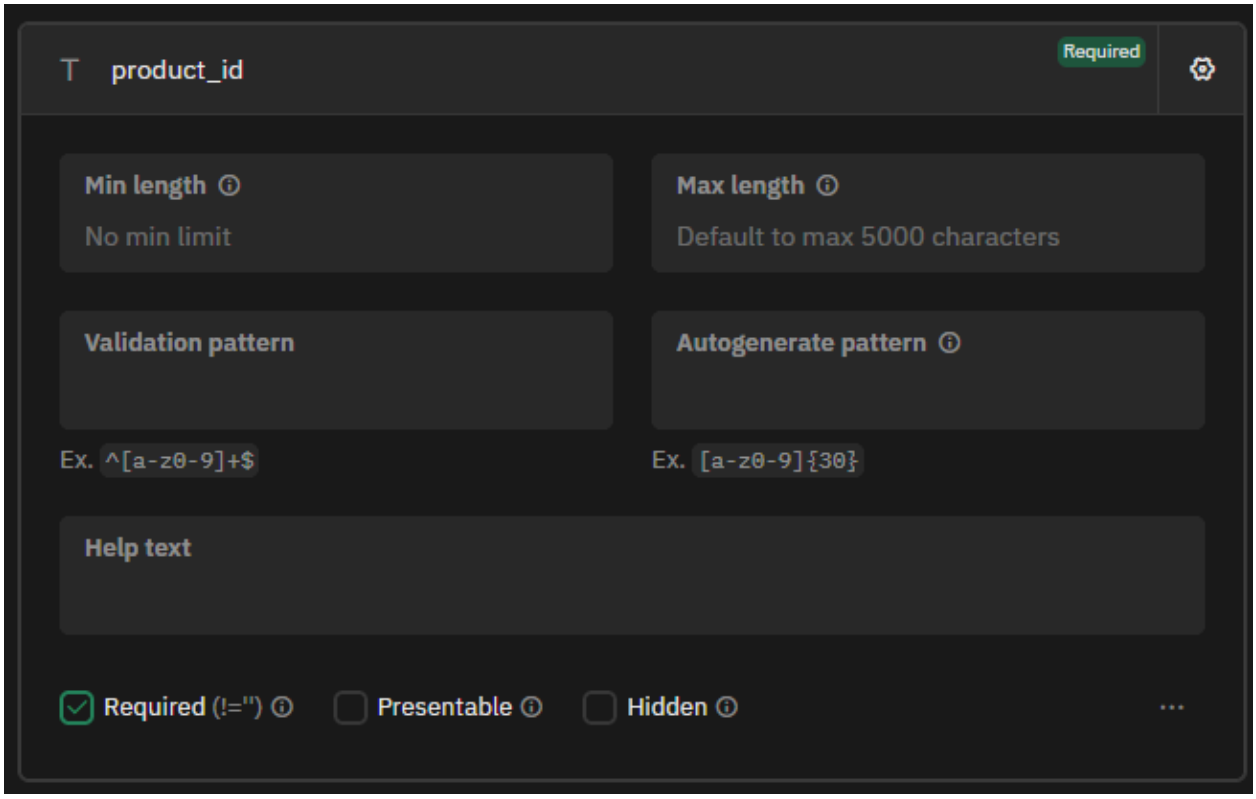
### 5. Power Bank 10000 mAh
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
Content-Type: application/json

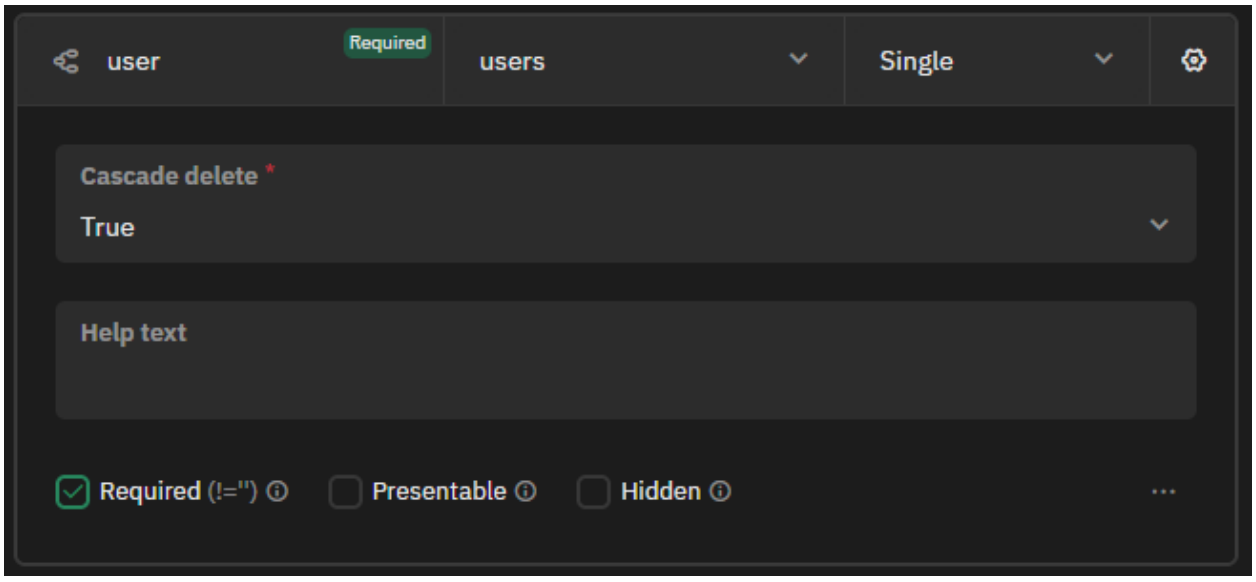
{
  "title": "Power Bank 10000 mAh",
  "description": "Pengisian cepat 20W, dua port USB.",
  "category": "electronics",
  "price": 15,
  "rating": 4.5,
  "stock": 50,
  "thumbnail": "https://picsum.photos/id/160/300"
}
```

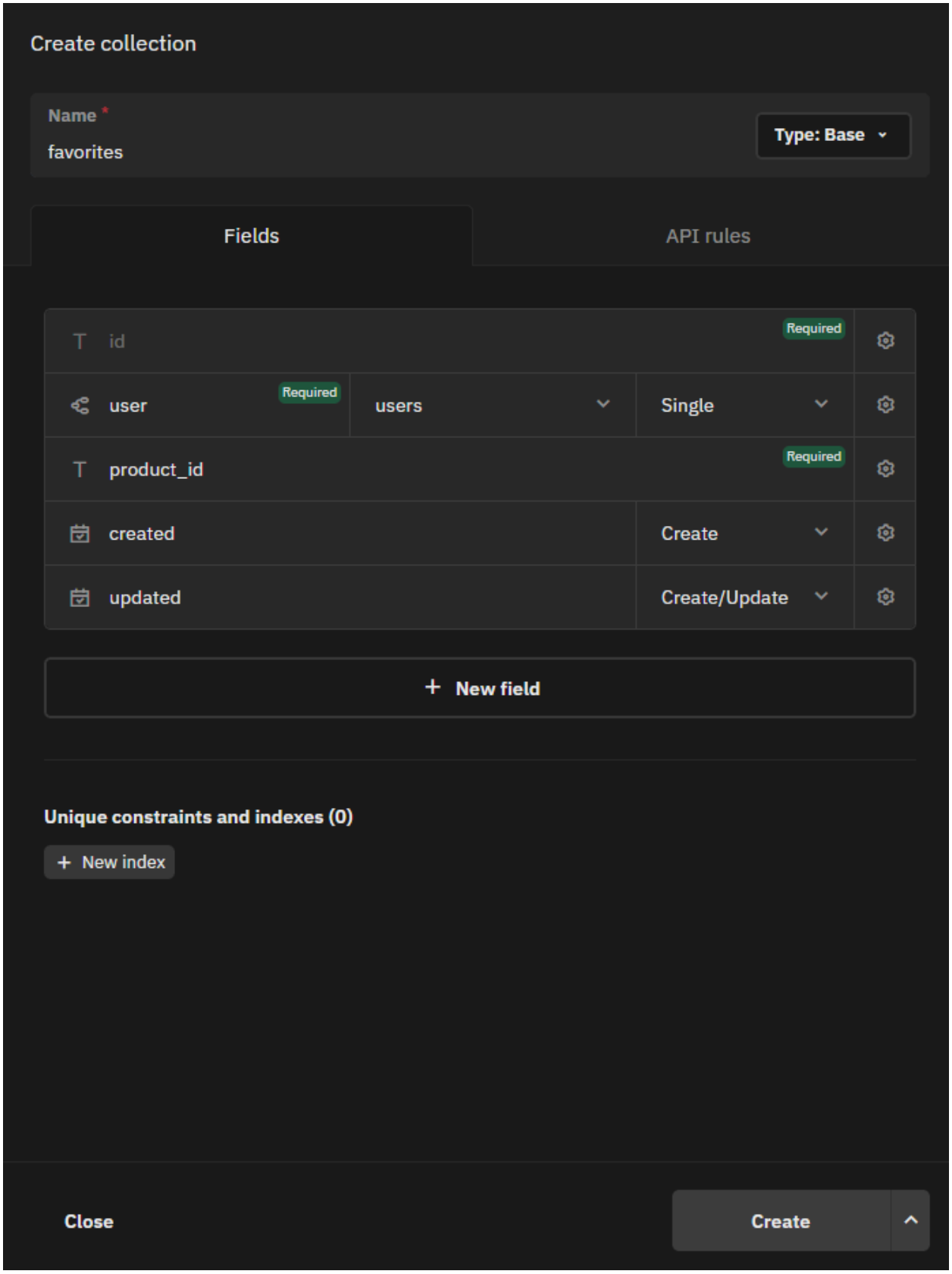


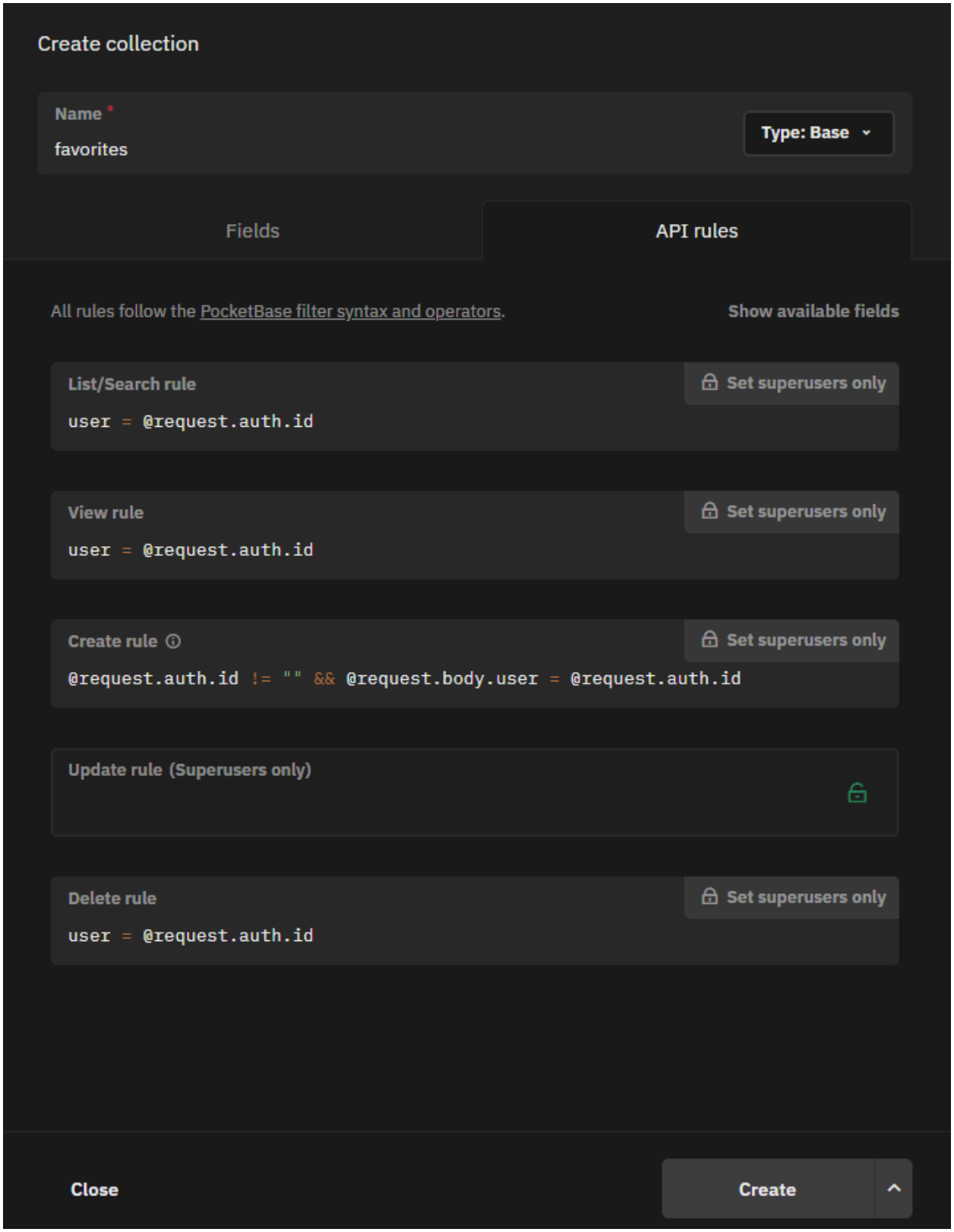
B11: account-service: users & favorites

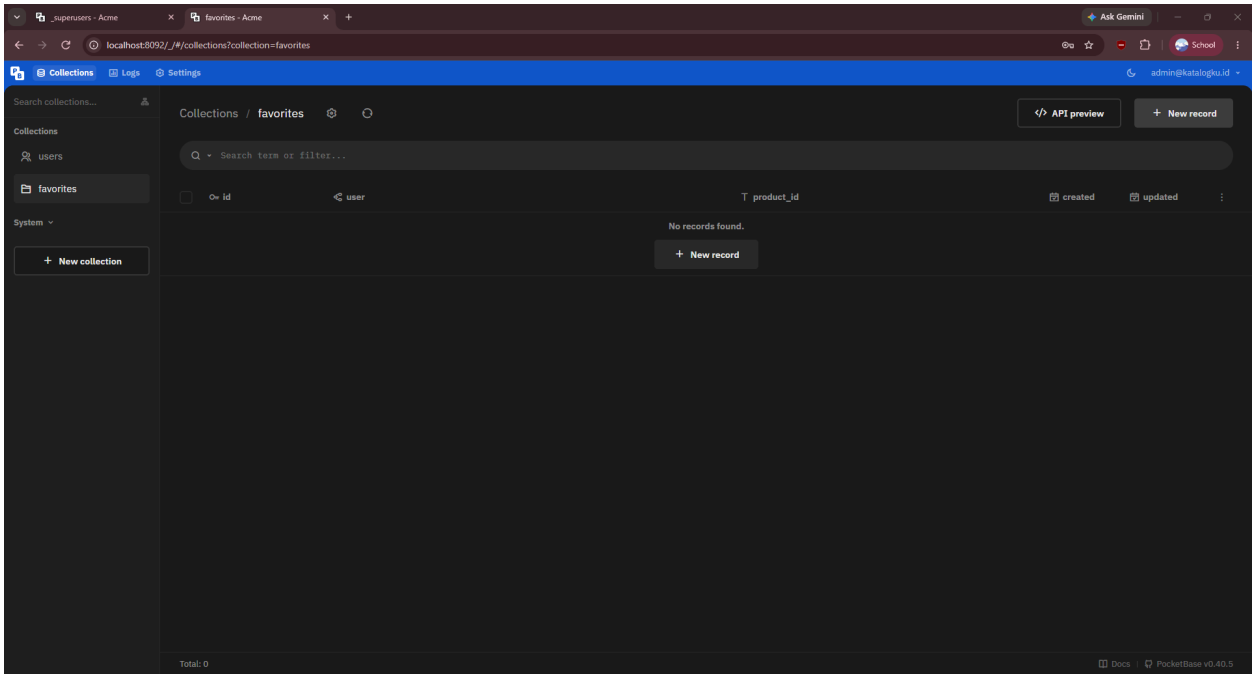



T product_id
Required
Min length
No min limit
Max length
Default to max 5000 characters
Validation pattern
Autogenerate pattern
Ex. ^[a-z0-9]+$
Ex. [a-z0-9]{30}
Help text
Required (!='')
Presentable
Hidden


user
Required
users
Single
Cascade delete *
True
Help text
Required (!='')
Presentable
Hidden


Create collection
Name *
favorites
Type: Base
Fields
API rules
id
Required
user
Required
users
Single
product_id
Required
created
Create
updated
Create/Update
+ New field
Unique constraints and indexes (0)
+ New index
Close
Create


Create collection
Name *
favorites
Type: Base
Fields
API rules
All rules follow the PocketBase filter syntax and operators.
Show available fields
List/Search rule
Set superusers only
user = @request.auth.id
View rule
Set superusers only
user = @request.auth.id
Create rule
Set superusers only
@request.auth.id != "" && @request.body.user = @request.auth.id
Update rule (Superusers only)
Delete rule
Set superusers only
user = @request.auth.id
Close
Create



B12: Kontrak API Gateway

Dalam file api.http untuk pengujian mekanisme

```
### Uji API KatalogKu (PocketBase) dengan ekstensi VS Code "REST Client"
### Klik "Send Request" di atas setiap request, berurutan 1-9.

# Ganti sesuai server: localhost / IP server / URL Cloudflare
@baseUrl = http://localhost:8080/api
# Nilai di bawah diambil otomatis dari hasil request sebelumnya
@productId = {{list.response.body.$.items[0].id}}
@token = {{login.response.body.$.token}}
@userId = {{login.response.body.$.record.id}}

### 1. Cek gateway
GET {{baseUrl}}/health

### 2. Daftar produk (5 terbaru)
# @name list
GET {{baseUrl}}/products?perPage=5&sort=-created

### 3. Cari produk dengan filter PocketBase
GET {{baseUrl}}/products?filter=title~'kopi'
```

```
### 4. Detail produk
GET {{baseUrl}}/products/{{productId}}

### 5. Daftar akun
POST {{baseUrl}}/auth/records
Content-Type: application/json

{
  "name": "Budi",
  "email": "budi@itb.ac.id",
  "password": "rahasia123",
  "passwordConfirm": "rahasia123"
}

### 6. Login
# @name login
POST {{baseUrl}}/auth/auth-with-password
Content-Type: application/json

{
  "identity": "budi@itb.ac.id",
  "password": "rahasia123"
}

### 7. Tambah favorit
POST {{baseUrl}}/favorites
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "user": "{{userId}}",
  "product_id": "{{productId}}"
}

### 8. Lihat favorit saya
# @name favs
GET {{baseUrl}}/favorites
Authorization: Bearer {{token}}
```

```
### 9. Hapus favorit pertama
DELETE {{baseUrl}}/favorites/{{favs.response.body.$.items[0].id}}
Authorization: Bearer {{token}}
```

Method	Endpoint	Auth	Body/Query	Hasil
GET	/api/health	-	-	200 Gateway aktif
GET	/api/products	-	?perPage=5&sort=-created	200 { items, pages, totalItems}
GET	/api/products	-	?filter=title~'kopi'	200 Hasil pencarian produk
GET	/api/products/:id	-	-	200 Detail produk / 404 tidak ditemukan
POST	/api/auth/records	-	name, email, password, passwordConfirm	200 Akun baru / 400 email sudah digunakan
POST	/api/auth/auth-with-password	-	identity (email), password	200 { token, record }
POST	/api/favorites	Bearer	User, product_id	200 Favorit ditambahkan / 400 jika sudah ada
GET	/api/favorites	Bearer	-	200 { items } Favorit milik pengguna
DELETE	/api/favorites/	Bearer	-	204 berhasil dihapus

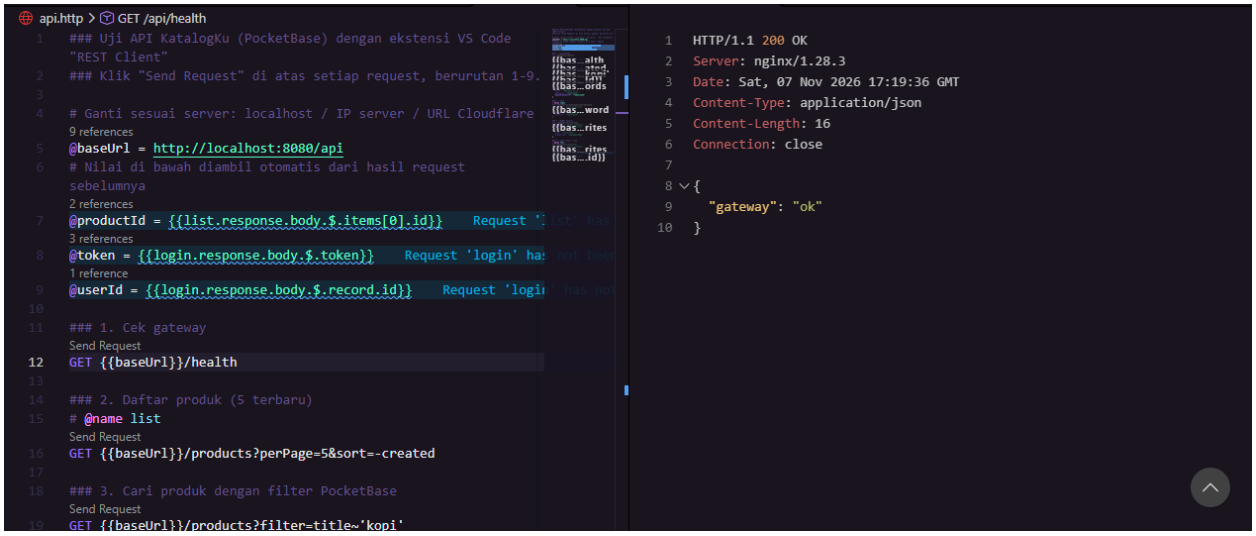
B13: Tools untuk Menguji API

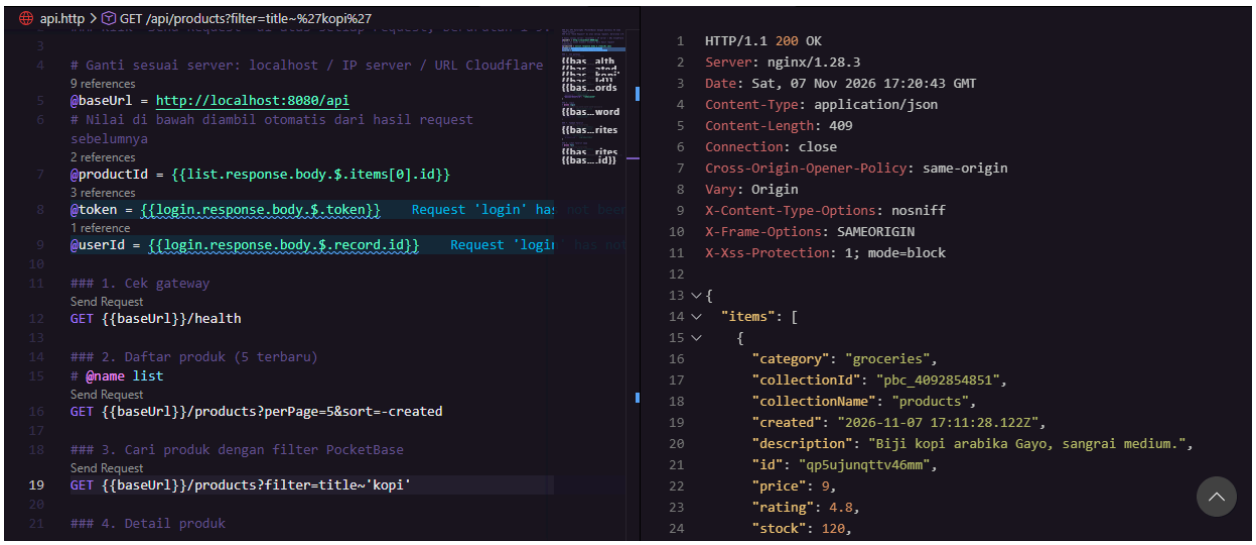
Tools yang digunakan pada pengujian API pada tugas ini yaitu.

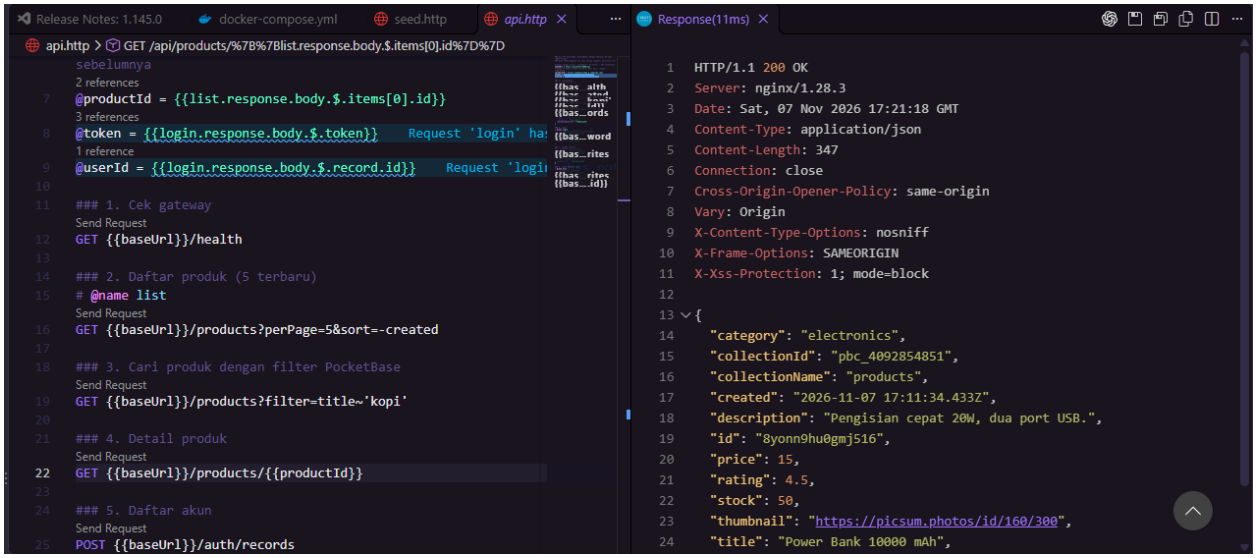
Alat	Kegunaan	Kenapa mudah

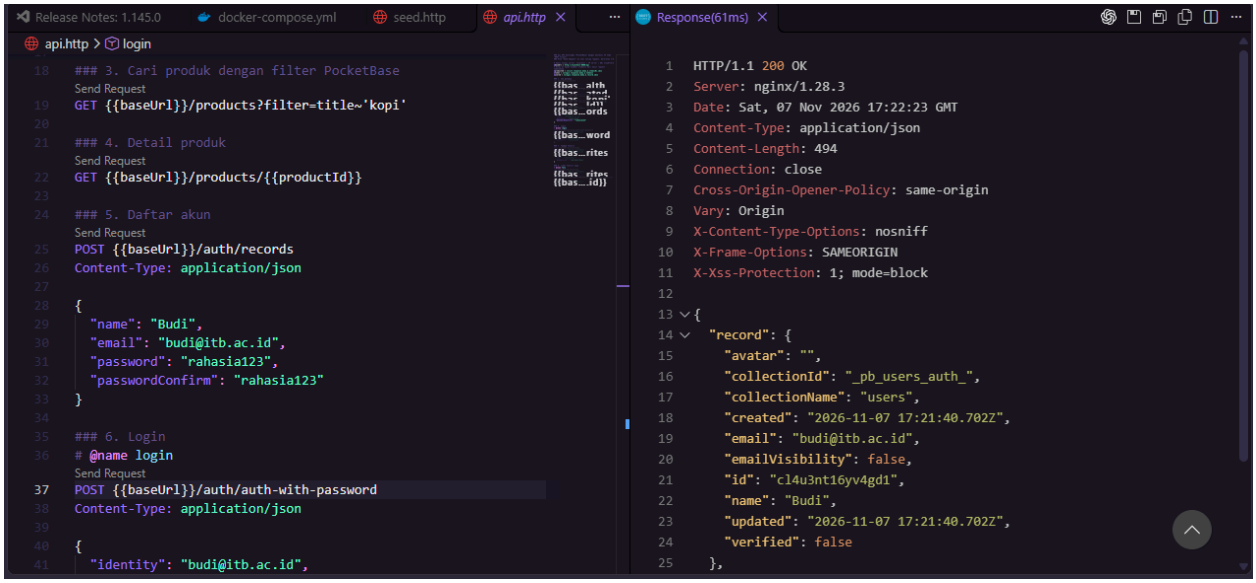
PocketBase Dashboard	Membuat koleksi, mengatur API Rules, serta melihat record.	Semua melalui browser, tanpa coding
VS Code + REST Client	Mengirim HTTP request melalui berkas api.http.	Gratis, tanpa akun tambahan, klik “Send Request”
Postman	Menjalankan koleksi permintaan dan memeriksa hasil pengujian.	Request dapat disimpan dan dikelompokkan dalam collection
Browser	Menguji cepat endpoint GET melalui alamat gateway.	Cukup ketik URL pada browser

B14: REST Client: Uji API dari VS Code






Release Notes: 1.145.0
docker-compose.yml
seed.http
api.http
Response(11ms)
api.http > GET /api/products/%7B%7Blist.response.body.$.items[0].id%7D%7D
sebelumnya
2 references
@productId = {{list.response.body.$.items[0].id}}
3 references
@token = {{login.response.body.$.token}} Request 'login' has
1 reference
@userId = {{login.response.body.$.record.id}} Request 'login
1. Cek gateway
Send Request
GET {{baseUrl}}/health
2. Daftar produk (5 terbaru)
@name list
Send Request
GET {{baseUrl}}/products?perPage=5&sort=-created
3. Cari produk dengan filter PocketBase
Send Request
GET {{baseUrl}}/products?filter=title~'kopi'
4. Detail produk
Send Request
GET {{baseUrl}}/products/{{productId}}
5. Daftar akun
Send Request
POST {{baseUrl}}/auth/records
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Sat, 07 Nov 2026 17:21:18 GMT
Content-Type: application/json
Content-Length: 347
Connection: close
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block
{
"category": "electronics",
"collectionId": "pbc_4092854851",
"collectionName": "products",
"created": "2026-11-07 17:11:34.433Z",
"description": "Pengisian cepat 20W, dua port USB.",
"id": "8yonn9hu0gmj516",
"price": 15,
"rating": 4.5,
"stock": 50,
"thumbnail": "https://picsum.photos/id/160/300",
"title": "Power Bank 10000 mAh",


Release Notes: 1.145.0
docker-compose.yml
seed.http
api.http
Response(61ms)
api.http > login
3. Cari produk dengan filter PocketBase
Send Request
GET {{baseUrl}}/products?filter=title~'kopi'
4. Detail produk
Send Request
GET {{baseUrl}}/products/{{productId}}
5. Daftar akun
Send Request
POST {{baseUrl}}/auth/records
Content-Type: application/json
{
"name": "Budi",
"email": "budi@itb.ac.id",
"password": "rahasia123",
"passwordConfirm": "rahasia123"
}
6. Login
@name login
Send Request
POST {{baseUrl}}/auth/auth-with-password
Content-Type: application/json
{
"identity": "budi@itb.ac.id",
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Sat, 07 Nov 2026 17:22:23 GMT
Content-Type: application/json
Content-Length: 494
Connection: close
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block
{
"record": {
"avatar": "",
"collectionId": "_pb_users_auth_",
"collectionName": "users",
"created": "2026-11-07 17:21:40.702Z",
"email": "budi@itb.ac.id",
"emailVisibility": false,
"id": "cl4u3nt16yv4gd1",
"name": "Budi",
"updated": "2026-11-07 17:21:40.702Z",
"verified": false
},

api.http > POST /api/favorites

```
# @name login
Send Request
POST {{baseUrl}}/auth/auth-with-password
Content-Type: application/json

{
  "identity": "budi@itb.ac.id",
  "password": "rahasia123"
}

### 7. Tambah favorit
Send Request
POST {{baseUrl}}/favorites
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "user": "{{userId}}",
  "product_id": "{{productId}}"
}

### 8. Lihat favorit saya
# @name favs
Send Request
GET {{baseUrl}}/favorites
Authorization: Bearer {{token}}

### 9. Hapus favorit pertama
```

```
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Sat, 07 Nov 2026 17:23:04 GMT
Content-Type: application/json
Content-Length: 215
Connection: close
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block

{
  "collectionId": "pbc_2151843437",
  "collectionName": "favorites",
  "created": "2026-11-07 17:23:04.374Z",
  "id": "2yxpsnrc19uglg5",
  "product_id": "8yonn9hu0gmj516",
  "updated": "2026-11-07 17:23:04.374Z",
  "user": "cl4u3nt16yv4gd1"
}
```

api.http > favs

```
# @name login
Send Request
POST {{baseUrl}}/auth/auth-with-password
Content-Type: application/json

{
  "identity": "budi@itb.ac.id",
  "password": "rahasia123"
}

### 7. Tambah favorit
Send Request
POST {{baseUrl}}/favorites
Authorization: Bearer {{token}}
Content-Type: application/json

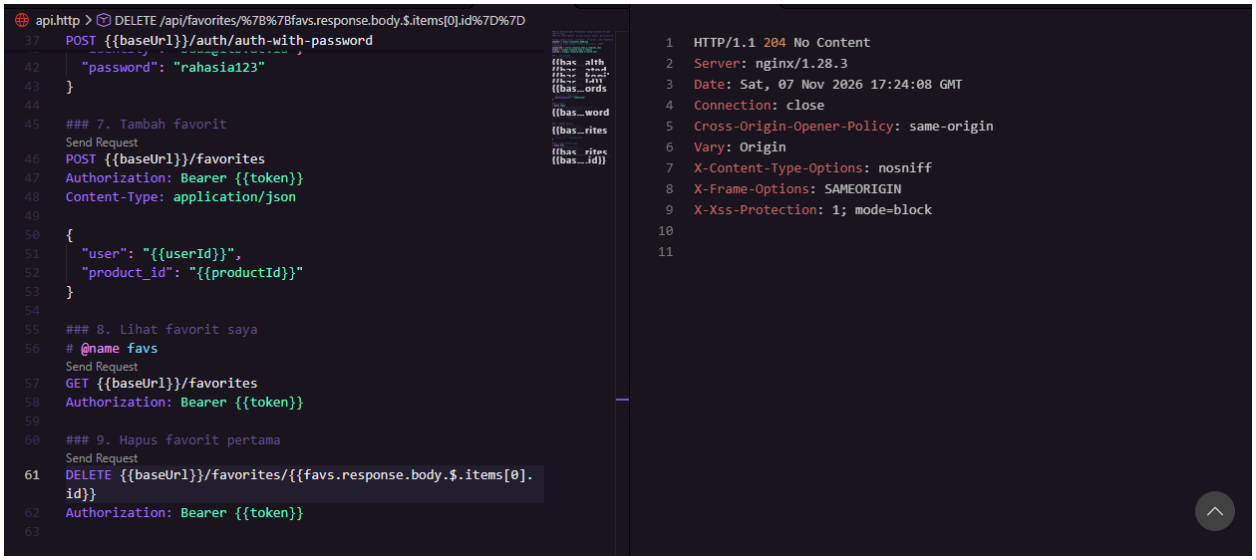
{
  "user": "{{userId}}",
  "product_id": "{{productId}}"
}

### 8. Lihat favorit saya
# @name favs
Send Request
GET {{baseUrl}}/favorites
Authorization: Bearer {{token}}

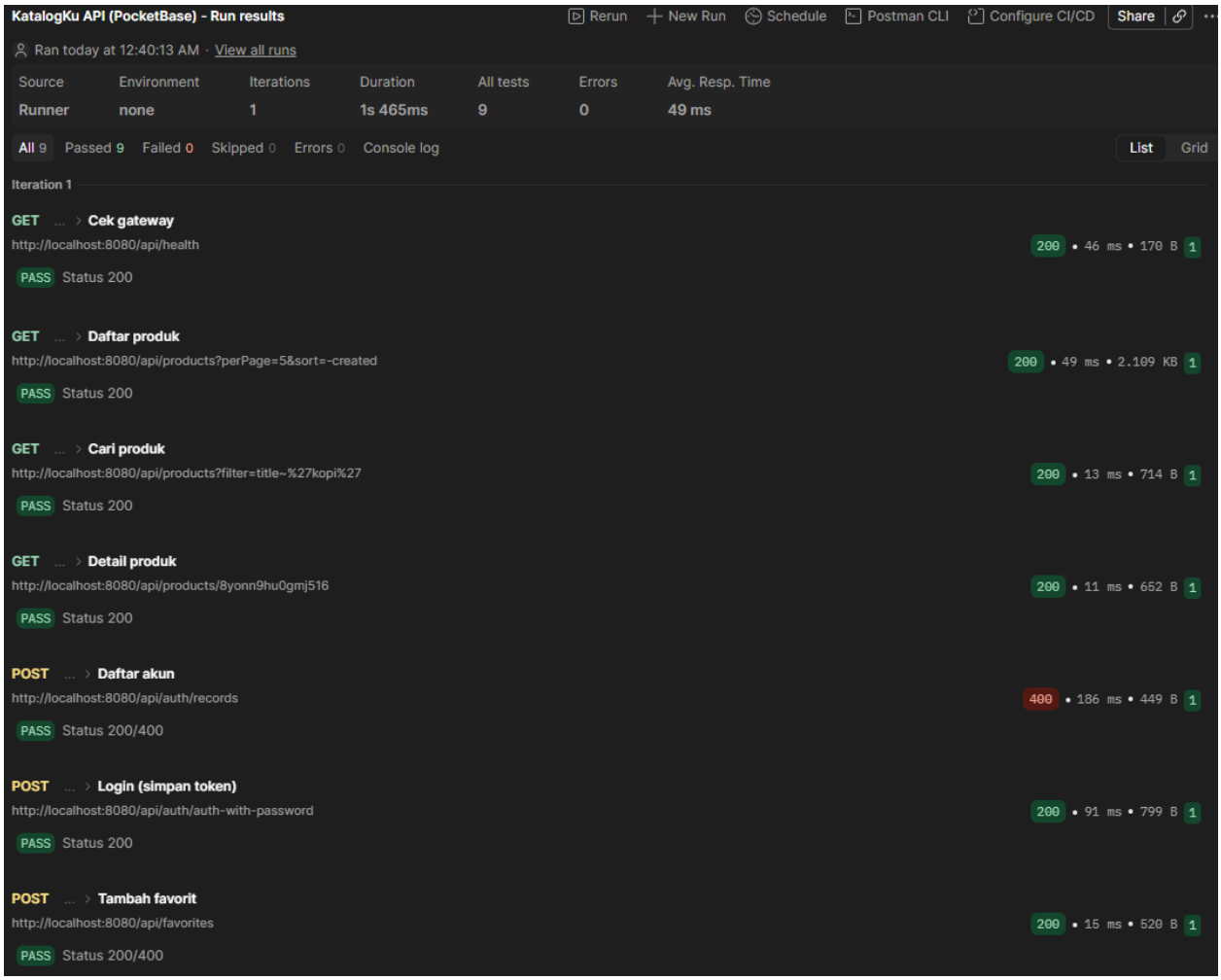
### 9. Hapus favorit pertama
```

```
HTTP/1.1 200 OK
Server: nginx/1.28.3
Date: Sat, 07 Nov 2026 17:23:46 GMT
Content-Type: application/json
Content-Length: 279
Connection: close
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block

{
  "items": [
    {
      "collectionId": "pbc_2151843437",
      "collectionName": "favorites",
      "created": "2026-11-07 17:23:04.374Z",
      "id": "2yxpsnrc19uglg5",
      "product_id": "8yonn9hu0gmj516",
      "updated": "2026-11-07 17:23:04.374Z",
      "user": "cl4u3nt16yv4gd1"
    }
  ],
  "page": 1,
```



B15: Postman: Import Koleksi Siap Pakai





B16: Alur Kerja Harian dengan PocketBase



Diagram di atas menunjukkan alur kerja harian menggunakan PocketBase berdasarkan materi TST Week 7. Pada tugas B, pengerjaan difokuskan pada konfigurasi layanan dan pengujian API menggunakan REST Client serta Postman. Tahapan selanjutnya, yaitu ekspor skema, commit, dan deployment ke server, dilanjutkan pada tugas C.